

BlockPro — `app_fetch_changes` Endpoint Specification

Purpose: Definitive spec for a new delta-sync workflow endpoint. This endpoint is the sole target of this document — everything else in the app stays unchanged.

1. Why this endpoint exists

Today the Flutter app syncs by calling `app_fetchbuildings`, then `app_fetch_all_assets` once per building, then `app_fetch_checklist_single` once per stale asset. Every refresh refetches **every** building and **every** asset, regardless of whether anything changed. The client uses `checklistlastmodified` inside the assets response to skip unchanged checklists, but there is no mechanism for:

- avoiding the full refetch of buildings and assets when nothing has changed.
- detecting a building or asset that has been **deleted** on the server,
- collapsing the N+1 round-trips for checklist refetches into a single response.

`app_fetch_changes` fixes all three. It returns everything that has changed for the authenticated user since a cursor the client supplies — buildings, assets, *and the full checklist content for any asset whose checklist has changed* — in a single call, including deletions.

This endpoint **supplements** the existing endpoints — it does not replace them. Initial sync (first login, or after a logout) continues to use `app_fetchbuildings` + `app_fetch_all_assets` + `app_fetch_checklist_single`. Every subsequent refresh uses `app_fetch_changes` and does **not** need to call `app_fetch_checklist_single` at all — the relevant checklist content is embedded directly in the delta response (see §6.2.4).

2. Endpoint summary

Property	Value
Path	<code>/wf/app_fetch_changes</code>
Method	GET
Auth	Bearer Token (required)
Content-Type (response)	application/json

3. Query parameters

Param	Type	Required	Description
<code>since</code>	string	No	Opaque cursor returned by a previous call to this endpoint. If omitted or empty, the server must return all buildings and assets for the user as added (acts as a fallback full sync).

The cursor is opaque to the client. The client stores it verbatim and hands it back next time. The server decides its format.

Recommended cursor format: an ISO 8601 UTC timestamp, e.g. "2026-04-22T09:15:03.412Z" . The server timestamp should be the value of "now" at the moment the response was generated, not the maximum `Modified Date` of the records in the response (see §7 for why this matters).

4. Response shape

The response body is a **real JSON object**, not a JSON-encoded string that needs a second parse.

Note for the Bubble developer: some existing endpoints (`app_fetchbuildings` , `app_fetch_all_assets` , `app_fetch_checkList_single`) return their payload as a JSON-encoded string — the client parses it twice. This new endpoint deliberately does **not** do that. The response `Content-Type` is `application/json` and the body is a directly parseable JSON object. The older behaviour is kept in the other endpoints for backward compatibility; please do not replicate it here.

```
{
  "status": "success",
  "response": {
    "cursor": "2026-04-22T09:15:03.412Z",
    "buildings": {
      "added": [ /* full Building objects */ ],
      "updated": [ /* full Building objects */ ],
      "deleted": [ "<buildingId>", "<buildingId>" ]
    },
    "assets": {
      "added": [ /* full Asset objects */ ],
      "updated": [ /* full Asset objects */ ],
      "deleted": [ "<assetId>", "<assetId>" ]
    }
  }
}
```

Top-level fields:

Field	Type	Required	Description
status	string	Yes	"success" on success; "error" on failure
response.cursor	string	Yes	New opaque cursor. Client stores this and sends it as since on the next call. Always present, even when all change lists are empty.
response.buildings.added	array of Building	Yes	Buildings that are newly visible to this user since since. Full records.
response.buildings.updated	array of Building	Yes	Buildings that existed before since but have been modified. Full records.
response.buildings.deleted	array of string	Yes	Bubble unique IDs of buildings no longer visible to this user (deleted, or access revoked). IDs only.
response.assets.added	array of Asset	Yes	Assets that are newly visible to this user since since. Full records.

response.assets.updated	array of Asset	Yes	Assets that existed before <code>since</code> but have been modified, or whose checklist has been modified . Full records.
response.assets.deleted	array of string	Yes	Bubble unique IDs of assets no longer visible (deleted, or their building access revoked). IDs only.

All six arrays must always be present, even if empty. The client expects to iterate over each unconditionally.

5. Object schemas

5.1 Building object

Identical to the Building object returned by `app_fetchbuildings` (see BlockPro-API-Spec-v2.md §5.2).

```
{
  "id": "1756484476534x490740256824968800",
  "name": "Building 1"
}
```

Field	Type	Description
id	string	Bubble unique ID of the building
name	string	Building display name

5.2 Asset object

All scalar fields are identical to the Asset object returned by `app_fetch_all_assets` (see BlockPro-API-Spec-v2.md §5.3). Use the exact same field names and types so the client can reuse the existing asset parser.

One addition for this endpoint: an optional `checklist` field, conditionally embedded when the checklist has changed. See §6.2.4 for the rules on when it must be present vs omitted.

```
{
  "taskname": "Fire door inspection",
  "assetnickname": "896",
  "assetId": "1771864899143x375085884294525250",
  "buildingId": "1756831363197x720244312004630400",
  "assetregisteritems": "",
  "tooltiptext": "",
  "tooltipurls": "",
  "lastcompleted": "2026-04-14T11:27:56.319Z",
  "duedate": "2026-04-14T11:27:56.319Z",
  "frequency": "7 Day(s)",
  "colour": "Red",
  "location": "",
  "floor": "",
  "yellowdate": "2026-04-14T11:27:56.319Z",
  "assetlastmodified": "2026-04-13T10:09:31.738Z",
  "checklistlastmodified": "2026-04-14T11:02:50.833Z",
}
```

```

"checklist": {
  "parentassetid": "1771864899143x375085884294525250",
  "chapters": [
    {
      "chaptername": "Fire door",
      "chapterorder": 1,
      "questions": [
        {
          "questiontext": "Does this door self-close fully into its frame?",
          "questiondesc": "",
          "answertype": "Yes|No",
          "photorequirement": "Always",
          "questionordernumber": 5,
          "questionid": "1771871963645x392498056957566000",
          "existingremedials": []
        }
      ]
    }
  ]
}

```

The two timestamp fields are critical and must be accurate:

Field	Meaning	How the client uses it
assetlastmodified	Last time any field on the Asset record itself changed (e.g. duedate, frequency, colour)	Stored on the asset row
checklistlastmodified	Effective modification date of the asset's full checklist — computed as described in §6.2.3	Stored on the asset row alongside the checklist content

These two timestamps are independent. An asset may appear in `assets.updated` because its `duedate` changed but its checklist did not — in which case the asset object will carry the new `duedate`, an unchanged `checklistlastmodified`, and **no `checklist` field**. Conversely, a checklist change must surface the asset in `assets.updated` with a moved `checklistlastmodified` **and** an embedded `checklist` field containing the full chapters/questions/remedials tree (see §6.2.4).

Shape of the embedded `checklist` object. Byte-for-byte identical to the object returned by `app_fetch_checklist_single` (see BlockPro-API-Spec-v2.md §5.5), with one exception: this endpoint returns it as a real JSON object (not a JSON-encoded string), consistent with §4. The client will reuse its existing `app_fetch_checklist_single` parser to consume it.

6. What counts as a "change"

6.0 Two independent sources feed this endpoint

Before reading the per-entity rules, understand that populating the response correctly requires **two independent mechanisms**:

1. **Record-level modification tracking** — `Modified Date` on the Building, Asset, and Checklist records. This drives `added` (when the record was created after `since`) and `updated`.

2. **Access-change tracking** — an audit of when this user gained or lost access to each building. This drives access-granted additions and access-revoked deletions.

A building's own `Modified Date` does not move when a user is added to it, and its own `Modified Date` does not move when it is soft-deleted (unless the soft-delete flag write bumps it — which is implementation-dependent). So **neither of the two mechanisms above is sufficient on its own**. Both must be wired up. This is called out again in §12 as an open question because the audit-table side may not exist yet in Bubble.

6.1 Buildings

Include a building in the response if any of the following happened after `since` :

Event	Bucket	Source
A building was created after <code>since</code> under a user the client already had access to (rare — most accounts don't auto-provision buildings)	<code>buildings.added</code>	Record <code>Modified Date</code> (creation is the first modification)
The user was granted access to a pre-existing building after <code>since</code>	<code>buildings.added</code>	Access-change audit
A building the user already had access to had any of its exposed fields (<code>name</code>) modified	<code>buildings.updated</code>	Record <code>Modified Date</code>
The building was deleted (hard or soft) after <code>since</code> and the user had access to it before	<code>buildings.deleted</code> (ID only)	Delete audit or soft-delete flag
The user lost access to a building after <code>since</code> (user removed, permission revoked)	<code>buildings.deleted</code> (ID only)	Access-change audit

"Exposed fields" means fields returned by this endpoint's Building schema. If Bubble-internal fields change but nothing in the returned schema changes, the building does **not** need to appear in `updated` . (A false positive here is harmless, just wasteful — it's fine to include it if filtering is cheaper to skip.)

6.2 Assets

Include an asset in the response if any of the following happened after `since` :

Event	Bucket	Source
A new asset was created under a building the user has access to	<code>assets.added</code>	Record <code>Modified Date</code>
The user gained access to a building, cascading to all of its assets	<code>assets.added</code>	Access-change audit (see §8.3)
An asset moved into a building the user has access to (rare — building reassignment)	<code>assets.added</code>	Record <code>Modified Date</code> + <code>buildingId</code> change
Any exposed field on the asset record changed (see §5.2 schema)	<code>assets.updated</code>	Asset record <code>Modified Date</code>
The asset's checklist was modified — any chapter, question, or existing remedial added, updated, or removed	<code>assets.updated</code> (with a fresh <code>checklistlastmodified</code>)	Checklist record <code>Modified Date</code> (see §6.2.1)

The asset was deleted (hard or soft), or the user lost access to its building	assets.deleted (ID only)	Delete audit, soft-delete flag, or access-change audit
---	--------------------------	--

A single asset must appear in **at most one** of the three buckets per response. If it was both added and updated within the same window, treat it as `added` (with the latest field values). If it was deleted and re-created under the same Bubble ID within one window (very unlikely but possible), treat it as `updated`.

6.2.1 The effective filter for assets

n.b. This is the single most common implementation mistake.

An asset must be surfaced in `updated` when **either** its own record **or any part of its checklist** has been modified since `since`. In pseudocode:

```
effective_modified = max(
  asset.Modified Date,
  checklist_last_modified(asset) // see §6.2.3 for how to compute this
)
if effective_modified > since:
  include in assets.updated
```

Two scenarios to keep in mind, both must work:

- 1. Asset changed, checklist unchanged.** Example: an inspection was completed (bumping `lastcompleted` and `duedate`), or the asset's `colour` / `frequency` / `location` changed. Caught by `asset.Modified Date > since`.
- 2. Asset unchanged, something inside the checklist changed.** Example: a question's wording was edited, a new question was added, a remedial was raised against a question. The asset record itself has not been touched. Caught only by `checklist_last_modified(asset) > since`.

An asset's own `Modified Date` does **not** move when only its checklist changes. If you filter only by `Asset.Modified Date > since`, every checklist-only change will be silently dropped. Do not do this. The filter must fold in the full checklist modification date as well — computed as described in §6.2.3.

The `checklistlastmodified` field in the emitted asset object must be the checklist's true current effective modification date, **not** the asset record's `Modified Date` and **not** the `Checklist` parent record's `Modified Date` in isolation (see §6.2.3).

6.2.2 `added` vs `updated` — client perspective

From the client's perspective, `added` and `updated` are semantically identical: both result in an upsert of the full asset record, followed by a comparison of `checklistlastmodified` against the stored value. The distinction is **informational only** and lets the server skip the "did this user already have this asset?" check for genuinely new records.

If classifying an asset as `added` vs `updated` is ambiguous on the Bubble side, default to `updated`. The client will do the right thing either way. Never put the same asset in both.

6.2.3 Computing `checklistlastmodified` — the cascade rule

`checklistlastmodified` is a **computed** value. It is **not** a single field you can read off the `Checklist` record directly, because changes to child records do not cascade their `Modified Date` up to the parent in Bubble.

Definition:

```
checklist_last_modified(asset) = max(
  Checklist.Modified Date      where Checklist.parent asset = asset,
  Chapter.Modified Date       for every Chapter   under that Checklist,
  Question.Modified Date      for every Question  under those Chapters,
  ExistingRemedial.Modified Date for every Remedial under those Questions
)
```

All four levels must be considered. If you read only `Checklist.Modified Date` , you will miss:

- A question's wording being edited.
- A question being added or deleted inside an existing chapter.
- A new chapter being added.
- A new existing remedial being raised against a question, or an existing one being edited or removed.

All of the above must bump `checklistlastmodified` . Right now the only way to guarantee that is to compute the `max` as shown.

Implementation options (pick whichever is cheapest on Bubble):

1. **Compute at read time.** On every `app_fetch_changes` call, for each candidate asset, compute the max across its chapters/questions/remedials. Simple but potentially N+1 — may be slow if a user has thousands of assets.
2. **Materialise on write.** On every write to a `Chapter` , `Question` , or `ExistingRemedial` , also update a single `checklistLastModified` field on the parent `Checklist` record. Then at read time, reading that one field is enough. Faster at read time, more write-side plumbing. This is the recommended approach if Bubble makes it easy to hook writes.

Either option is acceptable as long as the result is the true max across all four levels. If you pick option 2, remember to do the same for the `Asset.Modified Date` — a user completing an inspection needs to bump the asset record so the client picks up the new `lastcompleted` / `duedate` .

Note on existing remedials vs submitted inspection answers. "Existing remedials" in this context means the read-only remedial items that ship down with the checklist for display alongside questions (see BlockPro-API-Spec-v2.md §5.5). Inspection **answers** (what the user submits via `app_completed-inspection`) are not part of the checklist the client syncs — they are write-only from the client's perspective and do not need to appear in this endpoint's response.

6.2.4 When to embed the full checklist inline

To avoid the client making a follow-up round-trip to `app_fetch_checklist_single` for every asset whose checklist changed, this endpoint embeds the full checklist **inline** on the asset object, but only when the checklist has actually moved. The rules:

Bucket	Checklist change since since?	checklist field on emitted Asset
assets.added	n/a — this is a new asset, all of its content is new	Must be present (full chapters/questions/remedials tree)
assets.updated	Yes (<code>checklist_last_modified(asset) > since</code>)	Must be present (full chapters/questions/remedials tree)
assets.updated	No (only the asset record itself moved)	Must be omitted (or set to <code>null</code>)

<code>assets.deleted</code>	n/a — IDs only	n/a — the bucket contains strings, not Asset objects
-----------------------------	----------------	--

Why conditional embedding:

- If the checklist didn't change, the client already has the current content cached — sending it again is pure waste. The `checklistlastmodified` timestamp on the asset is the only thing that needs to move, and it won't (which is how the client knows to skip).
- If the checklist did change, the client will need the new content anyway, and inlining it avoids a second HTTP call per stale asset.

Shape of the embedded `checklist` : exactly the Checklist object schema from BlockPro-API-Spec-v2.md §5.5 — `parentassetid` plus `chapters[]` where each chapter has `chaptername` , `chapterorder` , `questions[]` , and each question carries its `existingremedials[]` . Unlike `app_fetch_checklist_single` , **do not wrap the checklist in a response envelope, do not return it as a JSON-encoded string, and do not wrap it in an outer single-element array**. It is a plain JSON object, nested directly on the asset.

Implementation note for `assets.added`. When the user is newly granted access to a building (§8.3), every asset under that building goes into `assets.added` and every one of those assets must carry its full `checklist` . For users gaining access to large building portfolios, this can produce a sizeable response — see §12 for the open question on pagination.

Implementation note for `assets.updated`. A common case will be an inspection being completed: `assetlastmodified` moves but the checklist does not. The asset still appears in `assets.updated` , but its `checklist` field is **omitted**. Do not include the checklist as a defensive "just in case" — the client relies on the field's absence to short-circuit the cache comparison.

6.3 Ordering within buckets

Order does not matter to the client. Return whatever is cheapest on the Bubble side.

6.4 Empty diff

If nothing has changed for the user since `since` , return all six arrays empty and a fresh `cursor` . This is the common case and must be cheap — target response time for the empty-diff case is under 500ms.

6.5 Referential invariant

For any response the server emits, the following must hold:

No asset in `assets.added` or `assets.updated` may have a `buildingId` that appears in `buildings.deleted` in the same response.

If the user loses access to a building, the building's ID goes in `buildings.deleted` and every one of its assets goes in `assets.deleted` (IDs only) — never in `added` / `updated` . This lets the client safely apply all deletions before applying all upserts without worrying about foreign-key dangling references.

7. Cursor semantics — please read carefully

The cursor is the single most important correctness property of this endpoint. Get it wrong and the client will silently miss changes.

7.1 What the cursor represents

The cursor represents "the point in server time up to which this user's changes have been delivered to this client." On the next call, the server must return every change with a `Modified Date > cursor` (or whatever equivalent marker you use internally).

7.2 How to generate the cursor (the critical algorithm)

Do this:

1. At the very start of handling the request, capture:
`T_now = server current UTC time`
`T = T_now - SAFETY_MARGIN` (see §7.3 for why)
2. Query for records modified in `(since, T]` (exclusive lower bound, inclusive upper)
3. Build the entire response using this single `T` value for every query in the request
4. Return `cursor = T` (not `T_now`, not `max(Modified Date)`)

Do not do any of these:

- `cursor = max(Modified Date)` across the records in the response — skips concurrent writes forever (explained in §7.3).
- `cursor = T_now` with no safety margin — skips records whose commit lands microseconds after the read (explained in §7.3).
- Calling `now()` a second time for a second query inside the same request — produces inconsistent windows across buildings vs assets.

7.3 Why the safety margin matters (read-write race at the high-watermark)

Consider this sequence with no safety margin:

```
T = 10:00:00.000 — request arrives, server captures T, runs SELECT
T = 10:00:00.005 — a transaction that started writing a record at 09:59:59.998
                    finally commits. Its Modified Date is 09:59:59.998.
                    Our SELECT at T=10:00:00.000 did NOT see it (uncommitted at read).
T = 10:00:00.010 — server returns cursor = 10:00:00.000 to the client.
— next call —
T = 10:00:05.000 — client calls again with since = 10:00:00.000
                    Server filters Modified Date > 10:00:00.000.
                    The record at 09:59:59.998 is LESS than since.
                    It is now permanently invisible.
```

The fix is simple: subtract a safety margin so committed-but-not-yet-visible writes at the cursor boundary get re-scanned on the next call.

Recommended: `SAFETY_MARGIN = 5 seconds`. This means the client may occasionally re-receive records it already has (harmless — they upsert to the same state), but no record is ever missed.

If you know your Bubble backend's longest possible write transaction latency and it's shorter than 5 seconds, you can reduce the margin. If in doubt, keep 5 seconds.

7.4 Inclusivity

The filter must be **exclusive** on the lower bound: `Modified Date > since`. The previous call already reported everything up to and including the previous `cursor` value. Using `>=` would double-report boundary records.

7.5 Single `T` per request

Capture `T` **once** at the start of the request and reuse the exact same value for:

- The buildings query
- The assets query
- The checklist-modification-date query (if it is a separate query)
- The cursor returned in the response

Calling `now()` a second time mid-request means a record modified between the first and second query can be double-reported or missed depending on which query sees it. A single `T` eliminates the hazard.

7.6 Clock source and skew

Use the Bubble server's clock as the single source of truth. Never trust a client-supplied timestamp. The client stores and replays whatever cursor the server gave it — the server never has to interpret a client-generated value for correctness.

If Bubble serves this endpoint from multiple processes or regions whose clocks may differ by a few hundred milliseconds, the safety margin from §7.3 also absorbs that skew. No further mitigation needed for skew $\leq$ `SAFETY_MARGIN` .

7.7 Clamp future `since` values

If the `since` value supplied by the client is **greater than the server's current time** (clock skew, tampered client, corrupted local DB, time-travel bug), clamp it:

```
effective_since = min(since, T_now)
```

Then proceed as normal. Do not error. A future `since` without clamping would filter out all legitimate records until server time catches up, silently breaking sync for that client.

7.8 Empty `since`

If `since` is absent, empty, or malformed (not parseable as an ISO 8601 timestamp), treat the request as a full sync: return every building and asset the user has access to in the `added` buckets, and return a fresh cursor. Do not error.

7.9 Timestamp format

All timestamps in both directions (the `since` parameter, the returned `cursor` , and every `*lastmodified` field in the payload) use:

- ISO 8601
- UTC (`Z` suffix, never a numeric offset)
- Millisecond precision

Example: `"2026-04-22T09:15:03.412Z"` .

7.10 Idempotency

Calling the endpoint twice with the same `since` must return logically equivalent data: the same set of record IDs in each bucket (subject to records modified between the two calls). The `cursor` may differ between the two calls (each call advances to a fresh `T`). A client that retries a call after a network failure must never end up with a corrupted state.

8. Authentication and user scoping

- Authentication is the standard `Authorization: Bearer <token>` header used by the rest of the app.
- The user identity is derived **entirely from the token. The endpoint does not accept a `userId` (or any equivalent) query parameter** — see §8.1 for why.
- "For this user" means every building the user currently has access to, plus every asset under those buildings. It does **not** include buildings the user used to have access to but no longer does — those buildings' IDs go into `buildings.deleted` (see §6.1).

8.1 Why there is no `userId` parameter

The Bubble dev will naturally ask: "different users have different access to different buildings — how does the endpoint know which user is calling?" The answer is: **the bearer token already identifies the user, and Bubble resolves it to `Current User` inside the workflow for free.** No parameter is needed.

Passing `userId` as a query parameter would be actively harmful for two reasons:

1. **Security — vertical privilege escalation.** If the endpoint honoured a client-supplied `userId`, any user could pass someone else's ID and receive that user's building and asset changes. Trusting only the token (which is cryptographically bound to the user who logged in) makes this impossible by construction.
2. **Ambiguity — two sources of truth.** If the token says user A but the `userId` param says user B, which wins? Either answer creates bugs. Having the token be the single source of truth removes the ambiguity entirely.

The existing `app_fetchbuildings` and `app_fetch_all_assets` endpoints already work this way — they take no `userId`, and they correctly return only the logged-in user's data because the Bubble workflow filters by `Current User`. The same pattern applies here.

8.2 Implementation hint for the Bubble workflow

Inside the backend workflow, reference the caller as `Current User's unique id` (and `Current User` for type lookups). Bubble's Workflow API auto-resolves the bearer token to `Current User` without any extra steps on your side — provided the following are true:

- The endpoint's **Authentication** setting is **"This endpoint requires authentication"**. Do **not** set it to "None required" / "Run without authentication" — that setting still accepts a valid bearer token but *also* permits anonymous calls in which `Current User` is empty, which would silently break user scoping.
- As a defensive first step in the workflow, assert `Current User is logged in`. If false, return a 401 per §9. This is cheap insurance against the endpoint accidentally being reconfigured to allow anonymous access in future.
- "Ignore privacy rules" is a separate setting and does not affect `Current User` resolution — set it according to your normal convention for internal app endpoints.

Do **not** implement custom token-to-user lookup logic. Bubble's built-in resolution is the idiomatic and correct approach, and it matches what the other `app_*` endpoints already do.

8.3 Access-change semantics

If a user is added to a building after `since`:

- The building appears in `buildings.added`.
- All of its assets appear in `assets.added`.

If a user is removed from a building after `since`:

- The building's ID appears in `buildings.deleted`.
- Every asset ID under that building appears in `assets.deleted`.

This keeps the client's local database consistent without needing a separate permissions-changed signal.

9. Error responses

Follow the same convention as the rest of the app (BlockPro-API-Spec-v2.md §6).

HTTP status	Condition	Response body
401	Missing, invalid, or expired bearer token	{ "status": "error", "message": "Unauthorized" }
500	Unexpected server error	{ "status": "error", "message": "<detail>" }

Do **not** return 400 for a missing or malformed `since` — treat those as a full sync (see §7.8). The endpoint should be forgiving about cursor input because clients may have lost their stored cursor (reinstall, DB corruption, etc.).

On error, the client will not advance its stored cursor, so the next successful call will resume from the same point. No data loss.

Client-side contract (not your concern to enforce, but worth understanding): the client will only persist the new `cursor` value **after** it has fully applied the response to its local database. If the response is received but apply fails (DB error, app killed mid-apply), the client keeps the old cursor and re-fetches the same window next time. This is why idempotency (§7.10) matters — the server may be asked to re-serve overlapping windows.

10. Example calls

10.1 Full sync fallback (no cursor)

Request:

```
GET /wf/app_fetch_changes
Authorization: Bearer <token>
```

Response:

```
{
  "status": "success",
  "response": {
    "cursor": "2026-04-22T09:15:03.412Z",
    "buildings": {
      "added": [
        { "id": "1756484476534x490740256824968800", "name": "Building 1" },
        { "id": "1756831363197x720244312004630400", "name": "Building 2" }
      ],
      "updated": [],
      "deleted": []
    },
    "assets": {
      "added": [
        {
          "taskname": "Fire door inspection",
          "assetnickname": "896",
          "assetId": "1771864899143x375085884294525250",
```

```

    "buildingId": "1756831363197x720244312004630400",
    "assetregisteritems": "",
    "tooltiptext": "",
    "tooltipurls": "",
    "lastcompleted": "2026-04-14T11:27:56.319Z",
    "duedate": "2026-04-14T11:27:56.319Z",
    "frequency": "7 Day(s)",
    "colour": "Red",
    "location": "",
    "floor": "",
    "yellowdate": "2026-04-14T11:27:56.319Z",
    "assetlastmodified": "2026-04-13T10:09:31.738Z",
    "checklistlastmodified": "2026-04-14T11:02:50.833Z",
    "checklist": {
      "parentassetid": "1771864899143x375085884294525250",
      "chapters": [
        {
          "chaptername": "Fire door",
          "chapterorder": 1,
          "questions": [
            {
              "questiontext": "Does this door self-close fully into its frame?",
              "questiondesc": "",
              "answertype": "Yes|No",
              "photorequirement": "Always",
              "questionordernumber": 5,
              "questionid": "1771871963645x392498056957566000",
              "existingremedials": []
            }
          ]
        }
      ]
    },
    "updated": [],
    "deleted": []
  }
}

```

Note the embedded `checklist` object on every `added` asset — see §6.2.4.

10.2 Nothing has changed

Request:

```

GET /wf/app_fetch_changes?since=2026-04-22T09:15:03.412Z
Authorization: Bearer <token>

```

Response:

```
{
  "status": "success",
  "response": {
    "cursor": "2026-04-22T10:42:18.907Z",
    "buildings": { "added": [], "updated": [], "deleted": [] },
    "assets": { "added": [], "updated": [], "deleted": [] }
  }
}
```

The cursor has advanced to the new "now", even though nothing changed.

10.3 Mixed changes

Request:

```
GET /wf/app_fetch_changes?since=2026-04-22T09:15:03.412Z
Authorization: Bearer <token>
```

Response:

```
{
  "status": "success",
  "response": {
    "cursor": "2026-04-22T11:03:50.000Z",
    "buildings": {
      "added": [],
      "updated": [
        { "id": "1756484476534x490740256824968800", "name": "Building 1 – renamed" }
      ],
      "deleted": [ "1756831363197x720244312004630400" ]
    },
    "assets": {
      "added": [],
      "updated": [
        {
          "taskname": "Fire door inspection",
          "assetnickname": "896",
          "assetId": "1771864899143x375085884294525250",
          "buildingId": "1756484476534x490740256824968800",
          "assetregisteritems": "",
          "tooltiptext": "",
          "tooltipurls": "",
          "lastcompleted": "2026-04-14T11:27:56.319Z",
          "duedate": "2026-04-21T11:27:56.319Z",
          "frequency": "7 Day(s)",
          "colour": "Yellow",
          "location": "",
          "floor": "",
          "yellowdate": "2026-04-14T11:27:56.319Z",
          "assetlastmodified": "2026-04-22T10:55:00.000Z",
          "checklistlastmodified": "2026-04-14T11:02:50.833Z"
        }
      ]
    }
  }
}
```

```

    },
    {
      "taskname": "Wet riser test",
      "assetnickname": "WR-1",
      "assetId": "1771999888777x111222333444555666",
      "buildingId": "1756484476534x490740256824968800",
      "assetregisteritems": "",
      "tooltiptext": "",
      "tooltipurls": "",
      "lastcompleted": "2026-03-01T09:00:00.000Z",
      "duedate": "2026-09-01T09:00:00.000Z",
      "frequency": "6 Month(s)",
      "colour": "Green",
      "location": "",
      "floor": "",
      "yellowdate": "2026-08-01T09:00:00.000Z",
      "assetlastmodified": "2026-03-01T09:00:00.000Z",
      "checklistlastmodified": "2026-04-22T10:30:00.000Z",
      "checklist": {
        "parentassetid": "1771999888777x111222333444555666",
        "chapters": [
          {
            "chaptername": "Pressure test",
            "chapterorder": 1,
            "questions": [
              {
                "questiontext": "Did the riser hold pressure at 12 bar for 15 minutes?",
                "questiondesc": "Reworded 2026-04-22 – previously asked about 10 bar.",
                "answertype": "Satisfactory|Unsatisfactory",
                "photorequirement": "Only when unsatisfactory",
                "questionordernumber": 1,
                "questionid": "1772000000001x111111111111111111",
                "existingremedials": []
              }
            ]
          }
        ]
      }
    }
  ],
  "deleted": [ "177000000000x111111111111111111" ]
}
}
}

```

In this example:

- Building 1 was renamed → appears in `buildings.updated` .
- Building 2 was deleted (or user removed from it) → appears in `buildings.deleted` .
- The first updated asset (Fire door) had its `duedate` and `colour` changed but **its checklist did not change** → `assetlastmodified` moved, `checklistlastmodified` unchanged, **no checklist field embedded**. The client updates the asset row and keeps its cached checklist (§6.2.4).

- The second updated asset (Wet riser) had its checklist reworted but the asset record itself is untouched → `assetlastmodified` is unchanged but `checklistlastmodified` has moved, **and the full checklist object is embedded inline**. The client updates the asset row, replaces the cached checklist with the embedded content, and makes **no** call to `app_fetch_checklist_single` .
 - One asset was deleted → ID in `assets.deleted` .
-

11. Checklist for the Bubble implementation

Endpoint basics

- ☐ Endpoint is registered at `/wf/app_fetch_changes` , GET, bearer-auth required.
- ☐ Response is a **real JSON object** with `Content-Type: application/json` , not a JSON-encoded string that requires a second parse (see §4 note).
- ☐ All six arrays (`buildings.added/updated/deleted` , `assets.added/updated/deleted`) are always present in the response, even when empty.

Cursor (§7) — the correctness-critical part

- ☐ Cursor `T` is captured **once** at the start of each request and reused for every query in that request.
- ☐ Cursor is generated as `T = now() - SAFETY_MARGIN` (recommended: 5 seconds), **not** `max(Modified Date)` and **not** `raw now()` .
- ☐ Filter on `Modified Date > since` (exclusive lower bound) and `<= T` (inclusive upper bound).
- ☐ `since` is optional; absent / empty / malformed is treated as a full sync, not an error (§7.8).
- ☐ `since` values greater than current server time are clamped to `now()` (§7.7).
- ☐ All returned timestamps are ISO 8601 UTC with `Z` suffix and millisecond precision (§7.9).

Entity rules (§6)

- ☐ **Asset filter folds in the checklist's effective modification date** — an asset must appear in `updated` when either the asset record or *anything in its checklist* has changed since `since` (§6.2.1). Do **not** filter only on `Asset.Modified Date` .
- ☐ `checklistlastmodified` is computed as `max(Checklist, Chapter, Question, ExistingRemedial) Modified Dates` (§6.2.3). Do **not** read only `Checklist.Modified Date` — a question-wording edit or a new remedial will not bump that field on its own.
- ☐ Editing a question's wording, adding/removing a question, adding a new chapter, and raising/editing/removing an existing remedial all surface the affected asset in `assets.updated` (even when the asset record itself is untouched).
- ☐ Completing an inspection (bumping the asset's `lastcompleted` / `duedate`) surfaces the asset in `assets.updated` via `assetlastmodified` .
- ☐ `assetlastmodified` reflects the asset record's `Modified Date` ; `checklistlastmodified` reflects the cascaded checklist max. They are populated independently.
- ☐ Building and Asset object shapes in `added` / `updated` are byte-for-byte identical to the shapes returned by `app_fetchbuildings` and `app_fetch_all_assets` respectively (plus the optional `checklist` field described next).
- ☐ **Embedded checklist field** (§6.2.4): every asset in `assets.added` carries a full `checklist` object. Every asset in `assets.updated` whose checklist changed carries a full `checklist` object. Every asset in `assets.updated` whose checklist did **not** change has the `checklist` field **omitted** (or `null`).
- ☐ Embedded `checklist` object shape matches `app_fetch_checklist_single` (§5.5 in the v2 spec) but is returned as a plain JSON object — no `response` envelope, no JSON-encoded string, no outer array.

- ☐ Deletions return IDs only (no full records).
- ☐ Access changes (user added to / removed from a building) are reflected as `added` / `deleted` on both buildings and the assets underneath — populated from the **access-change audit**, not from record `Modified Date` (§6.0).
- ☐ No asset in `assets.added` / `assets.updated` has a `buildingId` that appears in `buildings.deleted` in the same response (§6.5).
- ☐ When in doubt about `added` vs `updated` for an asset, default to `updated` (§6.2.2).

Performance

- ☐ Empty diff responds in under 500ms (§6.4).
- ☐ Endpoint is idempotent under client retries (§7.10).

12. Open questions for the Bubble developer

The client-side design of this endpoint is fixed, but the server-side implementation depends on three things we cannot determine from outside Bubble. Please confirm each before starting work:

1. **Soft delete vs hard delete.** How are deleted buildings/assets represented in the Bubble data model?
 - If deletes are **soft** (e.g. an `is_deleted` flag), the `.deleted` buckets can be populated by filtering for records where `is_deleted = true` and `Modified Date > since` — subject to the soft-delete write actually bumping `Modified Date`, please confirm it does.
 - If deletes are **hard**, a separate audit/log table is required to know what was removed. If this table does not exist today, it will need to be added before the endpoint can correctly report deletions.
 - Please describe what exists today so we can finalise §6.1 / §6.2.
2. **User-building membership history (access audit).** §6.0 and §8.3 require an audit trail of "user X gained/lost access to building Y at time T" to correctly populate `added` / `deleted` on access changes. Does such an audit exist in Bubble today? If not, adding it is a prerequisite for this endpoint.
3. **Checklist modification date accessibility.** §6.2.1 requires the endpoint to filter by `max(Asset.Modified Date, Checklist.Modified Date)`. Is the checklist's `Modified Date` easily joinable to the asset in a single Bubble query, or will this require N+1 lookups? If N+1 is unavoidable, flag it — we may need to rethink.
4. **Maximum response size.** Because this endpoint inlines the full `checklist` object on every `added` asset and on every `updated` asset whose checklist changed (§6.2.4), the response can get large in two specific scenarios:
 - A user is newly granted access to a portfolio with hundreds of buildings, each containing dozens of assets — every asset in `assets.added` carries its full checklist.
 - A bulk content update rewords questions across many checklists — every affected asset appears in `assets.updated` with its full checklist inlined.

For v1 we are **not** proposing pagination. Please flag if you expect the resulting single-response sizes (think: hundreds of assets × a few KB of checklist each = low-MB responses) to be a problem on the Bubble side or over mobile connections. If so, we'll add a `limit` / `continuation_token` pair in v2.